

第 1 章 関連研究

本研究は，(1) 視覚要素の配置を扱うレイアウト生成の研究，(2) 画像から物語的文脈を抽出・生成するビジュアルストーリーテリングの研究，(3) 漫画を対象とした研究，という 3 つの研究領域に関連し，本章ではこれらについて整理する．

まず，1.1 節でレイアウト生成研究の流れを概観し，要素配置問題としての定式化から，深層生成モデル，大規模言語モデルを用いた近年の発展までを整理する．次に，1.2 節で画像列から物語を生成するビジュアルストーリーテリング研究について，従来手法から大規模視覚言語モデルおよび Visual Prompting に至る技術的進展を述べる．最後に，1.3 節で漫画を対象とした画像理解・物語理解研究を取り上げ，特に Manga109Dataset を中心としたデータセット整備と，漫画特有の課題について整理する．

これらの関連研究を踏まえ，脚本に基づいて漫画のネームを生成するという本研究の位置づけと，既存研究に対する新規性を明確にする．

1.1 レイアウト生成

レイアウトの自動生成技術は，数理的な最適化に基づく古典的手法から，近年の深層学習を用いたデータ駆動型的手法へと大きく発展してきた．本節では，この技術的変遷を，体系的に記述するとともに，レイアウト生成タスクがどのような媒体を対象としてきたかを整理する．

1.1.1 ルールベースおよび最適化に基づくレイアウト生成

レイアウト生成の初期研究においては，視覚要素の配置を人間のデザイン知識に基づくルールや制約として明示的に定義し，それらを満たす配置を探索するアプローチが主流であった [**<empty citation>**].

このアプローチでは，レイアウト生成を「制約充足問題」あるいは「最適化問題」として定式化する．具体的には，要素の整列や間隔，余白の確保，サイズ比率の一貫性と

いったデザイン原則を数理的な制約条件として表現する．典型的には，各要素の位置 (x_i, y_i) や大きさ (w_i, h_i) を決定変数とし，可読性や審美性を反映した目的関数（エネルギー関数） E を定義して，その最大化（あるいはコストの最小化）を行うことでレイアウトを決定する [＜empty citation＞].

例えば，要素間の重なりを避ける制約，グリッドへの整列を促すペナルティ項，視線誘導を考慮した配置順序の制約などが導入され [＜empty citation＞]，これらを満たす最適解としてレイアウトが求められる．この枠組みは，専門家の知識を数式として表現する必要があったものの，レイアウト生成を明確な「要素配置問題」として定義した点で重要な基盤を与えた．

しかし，これらの手法は，設計者があらかじめ想定した制約や評価指標に強く依存するという本質的な課題を抱えていた．デザイン原則の重み付けは経験的に決定されることが多く，新たな媒体や異なる表現様式に適用する際にはルールの再設計が不可欠となる．また，複雑な制約を同時に満たす大域的最適解を探索することは計算量的に困難であり，生成される表現の多様性や柔軟性にも限界があった [＜empty citation＞].

こうした「人手による特徴設計」の限界から，データセットからレイアウトの分布や配置パターンを直接学習する手法への関心が高まり，次節で述べる深層生成モデルへと研究の主軸が移行していった．

1.1.2 深層生成モデルによるレイアウト生成

深層生成モデルは，データ分布を明示的あるいは暗黙的に近似することで，新たなサンプルを生成する枠組みである．レイアウト生成においては，視覚要素の配置全体を一つの確率分布として捉え，その分布から妥当な配置をサンプリングするという観点が重要となる．

GAN・VAE に基づく手法

ここでは，まず GAN および VAE の基本的定式化を整理した上で，それらをレイアウト生成に適用した代表的手法である LayoutGAN および LayoutVAE について詳述する．

Generative Adversarial Network (GAN) GAN [goodfellow2014generative] は，Generator G と Discriminator D の二者による敵対的学習を通じて，データ分布 $p_{\text{data}}(x)$ を暗黙的に近似する生成モデルである．Generator は潜在変数 $z \sim p(z)$ からデータ空間への写像 $G(z)$ を学習し，Discriminator は入力サンプルが実データか生成データかを識別する．学習は以下のミニマックス問題として定式化される：

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]. \quad (1.1)$$

GAN は高品質なサンプル生成が可能である一方、生成対象の構造を明示的に制御することが難しく、レイアウトのように「複数要素の相対関係」が重要なタスクでは、そのまま適用することは困難であった。

LayoutGAN LayoutGAN [li'layoutgan'2019] は、GAN をレイアウト生成に適用するため、レイアウトを複数要素の集合として定式化した。各レイアウトは、

$$\mathcal{L} = \{(b_i, c_i)\}_{i=1}^N \quad (1.2)$$

として表され、 $b_i = (x_i, y_i, w_i, h_i)$ は要素 i の位置およびサイズ、 c_i はカテゴリラベルである。

Generator G は、潜在変数 z を入力として、 N 個の要素特徴 $\{f_i\}_{i=1}^N$ を生成し、それぞれから (b_i, c_i) を出力する。しかし、単純に独立生成すると要素間の整列や重なりといった構造的関係を捉えられない。そこで LayoutGAN は、要素間の相互作用を考慮するために *relation module* を導入している。

この module では、要素 i と j の関係を相対位置などの関係特徴 r_{ij} として表し、注意機構に類似した計算により要素特徴を更新する：

$$\tilde{f}_i = \sum_{j=1}^N \alpha_{ij} f_j, \quad (1.3)$$

$$\alpha_{ij} = \frac{\exp(\phi(f_i, f_j, r_{ij}))}{\sum_{k=1}^N \exp(\phi(f_i, f_k, r_{ik}))}, \quad (1.4)$$

ここで $\phi(\cdot)$ は学習可能な関数である。この設計により、LayoutGAN は要素集合全体を一つの構造として扱い、整合性のあるレイアウト生成を可能にしている。

Variational Autoencoder (VAE) VAE [kingma2013autoencoding] は、確率的潜在変数モデルに基づき、データ x を潜在変数 z を介して生成する枠組みである。Encoder $q_\phi(z | x)$ と Decoder $p_\theta(x | z)$ を用い、対数尤度 $\log p_\theta(x)$ の下界である変分下限 (ELBO) を最大化する：

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)). \quad (1.5)$$

VAE は、潜在空間の連続性と生成の安定性に優れる一方、生成結果が平均化されやすいという特徴を持つ。

LayoutVAE LayoutVAE [jyothi2021layoutvaestochasticscenelayout] は、VAE をレイアウト生成に適用し、要素数が可変なレイアウトを確率的に生成可能とした。レイアウト $\mathcal{L}$ を、

$$\mathcal{L} = \{(e_i, b_i, c_i)\}_{i=1}^N \quad (1.6)$$

として表し、 $e_i \in \{0, 1\}$ は要素 i の存在を表す確率変数である。Decoder は以下のように分解される：

$$p_{\theta}(\mathcal{L} | z) = \prod_{i=1}^N p(e_i | z) p(b_i | z, e_i) p(c_i | z, e_i). \quad (1.7)$$

この定式化により、LayoutVAE はレイアウト構造そのものの不確実性を潜在変数として扱うことが可能となった。また、複数の妥当なレイアウトを確率的に生成できる点が特徴である。

Transformer に基づくレイアウト生成

GAN や VAE に基づく手法では、レイアウト全体を一括して生成することにより、要素間の幾何学的関係を暗黙的に学習するアプローチが取られてきた。一方で、要素数が可変であること、要素間に明示的な制約（関係・サイズ・配置条件など）が存在すること、また部分的な入力からの生成（補完・洗練）といった多様な条件付き生成を単一の枠組みで扱うことは困難であった。

この課題に対し、近年ではレイアウトを「要素列」として扱い、自己注意機構により要素間関係を明示的にモデル化する Transformer ベースの手法が提案されている。代表的な手法として、LayoutTransformer [gupta'layouttransformer'2021], BLT [kong2021blt], LayoutFormer [jiang'layoutformer'2023], およびその拡張である LayoutFormer++ [jiang'layoutformerpp'2023] が挙げられる。

Transformer の基本構造と系列モデリング Transformer は、自己注意機構（Self-Attention）により入力系列全体を同時に参照することで、長距離依存関係を効率的に捉えることができるモデルである。レイアウト生成においては、各要素を系列中のトークンとして扱うことで、要素間の複雑な相互作用を自然にモデル化できる点が重要である。

入力系列を $X = (x_1, x_2, \dots, x_T)$ とし、各トークン x_t を d 次元のベクトル表現 $h_t \in \mathbb{R}^d$ に埋め込む。これらをまとめた行列を $H \in \mathbb{R}^{T \times d}$ とする。自己注意機構では、 H から線形変換によりクエリ Q 、キー K 、バリュー V を構成する：

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \quad (1.8)$$

ここで, $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ は学習可能な重み行列である.

注意重みは, スケールド内積に基づき次式で計算される:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (1.9)$$

この操作により, 各要素は他のすべての要素を参照し, 文脈 (レイアウト全体の構造) を考慮した表現へと更新される.

さらに, 複数の注意ヘッドを並列に用いる Multi-Head Attention (MHA) により, 異なる部分空間における要素間関係を同時に学習する:

$$\text{head}_i = \text{Attention}(HW_Q^{(i)}, HW_K^{(i)}, HW_V^{(i)}), \quad (1.10)$$

$$\text{MHA}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O. \quad (1.11)$$

続いて, 各位置ごとに Position-wise Feed-Forward Network (FFN) が適用される:

$$\text{FFN}(h) = \sigma(hW_1 + b_1)W_2 + b_2, \quad (1.12)$$

ここで $\sigma(\cdot)$ は ReLU や GELU などの活性化関数である. これらの層が積層されることで, モデルはレイアウト要素間の高次な依存関係を獲得する.

レイアウトの系列化表現 上述の Transformer アーキテクチャをレイアウト生成に適用するためには, 2 次元的なレイアウト情報を 1 次元のトークン列に変換する必要がある. Transformer 系手法では, 一般にレイアウトを N 個の要素から構成される系列として表現する.

各要素 i は要素種別 c_i と, 位置および大きさを表す属性 (x_i, y_i, w_i, h_i) により定義される. これらの連続値は事前に量子化され, 各属性が離散トークンとして扱われる. このとき, レイアウト L は次のような系列となる:

$$L = \langle \text{SOS} \rangle c_1, x_1, y_1, w_1, h_1, \dots, c_N, x_N, y_N, w_N, h_N \langle \text{EOS} \rangle. \quad (1.13)$$

Transformer デコーダは, この系列を自己回帰的に生成し, 時刻 t におけるトークン L_t の生成確率を

$$P(L_t \mid L_{<t}, S; \theta) \quad (1.14)$$

としてモデル化する. ここで S は条件情報 (制約) を表し, θ はモデルパラメータである.

学習は, 負の対数尤度を最小化することで行われる:

$$\mathcal{L} = - \sum_{t=1}^{|L|} \log P(L_t \mid L_{<t}, S; \theta). \quad (1.15)$$

LayoutFormer++ による制約直列化 レイアウトの生成においてはその制御性を高めるために様々な制約を入力として与えることが試みられてきた．要素種別，要素サイズ，要素間関係（上下・左右・大小関係）など，多様な制約である．既存研究では，制約をレイアウトと同一形式で与える方法や，制約ごとに専用のモデル設計を行う方法が取られてきたが，これらは柔軟性に欠け，新たな制約設定への拡張が困難であった．Transformer ベースの手法である LayoutFormer++ は，これらの課題に対し，制約直列化（**constraint serialization**）という枠組みを導入した．これは，異なる種類の制約をすべて「トークン列」として統一的に表現する手法である．

例えば，要素種別制約，サイズ制約，要素間関係制約は，それぞれ次のような系列として表現される：

$$S_{\text{type}} = \langle \text{SOS} \rangle c_1 | c_2 | \cdots | c_N \langle \text{EOS} \rangle, \quad (1.16)$$

$$S_{\text{size}} = \langle \text{SOS} \rangle c_1 w_1 h_1 | \cdots | c_N w_N h_N \langle \text{EOS} \rangle, \quad (1.17)$$

$$S_{\text{rel}} = \langle \text{SOS} \rangle c_i i r_{i,j} c_j j | \cdots \langle \text{EOS} \rangle. \quad (1.18)$$

これらを一定の順序で連結することで，複数制約を同時に扱う入力系列 S が構成される．この設計により，LayoutFormer++ はレイアウト補完，洗練，関係条件付き生成など，異なるタスクを単一の Transformer モデルで統一的に扱うことが可能となった．

制約付きデコード (Decoding Space Restriction) さらに LayoutFormer++ は，推論時において **制約付きデコード** と呼ばれる戦略を導入している．各時刻 t において，Transformer デコーダが出力する確率分布 P_t に対し，以下の二段階の剪定を行う：

1. 制約違反を確実に引き起こすトークンの除外
2. 低確率トークンの除外

この結果得られる制限付き分布 P'_t からトークンをサンプリングすることで，生成品質を維持しつつ，制約違反の発生を抑制する．また，すべての候補が除外された場合には，過去の時刻に戻るバックトラッキング機構を導入し，生成の安定性を確保している．

拡散モデルによるレイアウト生成

一方，系列モデルである Transformer とは異なる生成パラダイムとして，近年では拡散モデル（Diffusion Models; DMs）を用いたアプローチも台頭している．拡散モデル（Diffusion Models; DMs）は，データ分布からの生成を「ノイズ付加による前向き

過程」と「ノイズ除去による逆過程」の逐次推定として定式化する生成モデルである。近年の画像生成で顕著な成功を収めた枠組みを背景に、レイアウト生成においても、複雑な要素間依存を段階的に回復するという性質が有効に働くことが示されている。

連続拡散 (DDPM) の基本定式化 まず連続値を対象とする代表的枠組みとして、Denoising Diffusion Probabilistic Models (DDPM) を概観する。データ $x_0 \sim q(x_0)$ に対し、前向き過程 $q(x_{1:T} | x_0)$ は段階的にガウスノイズを付与するマルコフ連鎖として定義される：

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (1.19)$$

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1}), \quad (1.20)$$

ここで $\{\beta_t\}_{t=1}^T$ はノイズスケジュールである。このとき x_t は閉形式で

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s \quad (1.21)$$

と書けるため、

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (1.22)$$

として直接サンプリングできる。

生成は逆過程 $p_\theta(x_{0:T})$ を学習することで行い、一般に

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=T}^1 p_\theta(x_{t-1} | x_t), \quad p(x_T) = \mathcal{N}(0, I) \quad (1.23)$$

と定義される。逆遷移 $p_\theta(x_{t-1} | x_t)$ はガウス分布で近似され、平均 $\mu_\theta(x_t, t)$ (あるいはノイズ予測器 $\epsilon_\theta(x_t, t)$) を学習する。典型的にはノイズ予測損失

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2] \quad (1.24)$$

が用いられ、 T ステップの反復により $x_T \rightarrow x_0$ を復元することで生成を行う。

レイアウト生成における「離散拡散」への動機 レイアウトは、要素カテゴリ c_i と量子化された座標・サイズ (x_i, y_i, w_i, h_i) のように、しばしば離散値 (トークン) として表現されるのは前述の通り。そのため、連続値ガウス拡散 (DDPM) を直接適用するよりも、語彙集合 $\mathcal{V}$ 上のカテゴリ分布遷移としてノイズ付加を定義する**離散拡散 (discrete diffusion)** を用いる設計が自然となる。

離散拡散では、各時刻 t の状態 z_t を $K = |\mathcal{V}|$ 通りのカテゴリ変数とし、前向き過程 (forward process) をマルコフ連鎖として

$$q(z_{1:T} | z_0) = \prod_{t=1}^T q(z_t | z_{t-1}) \quad (1.25)$$

で定義する。D3PM (Discrete Denoising Diffusion Probabilistic Models) 等の枠組みでは、各ステップの遷移は遷移行列 $Q_t \in \mathbb{R}^{K \times K}$ を用いて

$$q(z_t | z_{t-1}) = \text{Cat}(z_t; z_{t-1} Q_t) \quad (1.26)$$

と書ける (ここで z_{t-1} は one-hot とみなす)。このとき、 t ステップ後の周辺分布は累積遷移 $\bar{Q}_t := \prod_{s=1}^t Q_s$ を用いて

$$q(z_t | z_0) = \text{Cat}(z_t; z_0 \bar{Q}_t) \quad (1.27)$$

と表される。

逆過程は、ニューラルネットワークによる条件付き分布

$$p_\theta(z_{t-1} | z_t) = \text{Cat}(z_{t-1}; \pi_\theta(z_t, t)) \quad (1.28)$$

で近似され、 $z_T \rightarrow z_0$ を段階的に復元することで生成を行う。さらに、変分下限 (VLB) に基づく学習では、各時刻での posterior $q(z_{t-1} | z_t, z_0)$ が重要となる。マルコフ性とベイズ則から、

$$q(z_{t-1} | z_t, z_0) \propto q(z_t | z_{t-1}) q(z_{t-1} | z_0) \quad (1.29)$$

であり、行列表現を用いると (one-hot の積として)

$$q(z_{t-1} | z_t, z_0) \propto (z_t Q_t^\top) \odot (z_0 \bar{Q}_{t-1}) \quad (1.30)$$

の形で計算できる ($\odot$ は要素積)。レイアウトではさらに、(1) カテゴリ (要素種別) のような非順序 (nominal) 属性と、(2) 座標・サイズのような順序 (ordinal) 属性が混在するため、前向き遷移 Q_t の設計が生成品質を大きく左右する点が重要である。

LayoutDM: 離散状態空間拡散による統一的レイアウト生成 LayoutDM [inoue2023layoutdm]

は、レイアウトを離散表現として扱い、離散状態空間拡散 に基づく単一モデルで多様なレイアウト生成タスクを統一的に扱う枠組みを提案した。基本的な発想は、レイアウトを構成する属性 (カテゴリ, 座標, サイズなど) をそれぞれ語彙集合上のカテゴリ変数として扱い、属性 (modality) ごとに拡散ノイズを設計する点にある。

(i) マスク置換型 **forward** 遷移の定式化 LayoutDM は D3PM 系の離散拡散をベースとし、語彙 $\{1, \dots, K\}$ に加えて特殊トークン MASK を導入した **mask-and-replace** の前向き遷移を用いる。各ステップの遷移行列 Q_t を

$$Q_t = \begin{bmatrix} (1 - \beta_t)I_K & \beta_t \mathbf{1}_K \\ \mathbf{0}_K^\top & 1 \end{bmatrix}, \quad (1.31)$$

のように定義することで、通常トークンは確率 β_t で MASK に遷移し、MASK は吸収状態 (absorbing state) として保持される。ここで I_K は $K \times K$ 単位行列、 $\mathbf{1}_K$ は長さ K のベクトル (MASK への遷移成分を表す) である。この設計により、カテゴリを他カテゴリへランダムに崩すのではなく、「不確かさ」を MASK として明示的に表現できるため、構造化データ (レイアウト) に対して安定なノイズ付加が可能となる。

(ii) 逆過程のパラメータ化 (**posterior mixing**) 逆過程では、Transformer Encoder 等のネットワークで $\tilde{p}_\theta(\tilde{z}_0 | z_t)$ (元トークンの事後分布) を推定し、これを posterior $q(z_{t-1} | z_t, \tilde{z}_0)$ と混合して

$$p_\theta(z_{t-1} | z_t) \propto \sum_{\tilde{z}_0} q(z_{t-1} | z_t, \tilde{z}_0) \tilde{p}_\theta(\tilde{z}_0 | z_t) \quad (1.32)$$

と構成する。これは「 z_0 を直接当てる」予測器を用いつつ、拡散過程に整合した z_{t-1} の分布へ落とし込む D3PM の代表的パラメータ化であり、VLB に基づく学習 (KL 項の最小化) と相性が良い。

(iii) 条件付き生成 (制約注入) の数式化 LayoutDM は、条件付き生成を「再学習」ではなく推論時操作として扱える点を特徴とする。例えば、既知トークン集合 Ω (部分レイアウト、固定属性など) に対し、逆過程サンプリングの確率を

$$p'_\theta(z_{t-1} | z_t) \propto p_\theta(z_{t-1} | z_t) \prod_{j \in \Omega} \mathbb{I}[z_{t-1}^{(j)} = \hat{z}^{(j)}] \quad (1.33)$$

のように「固定 (masking)」することで、指定部分を常に満たすサンプリングが可能になる。また、制約違反候補の logit を減衰させる logit adjustment は、

$$\log \pi'_\theta(v | z_t, t) = \log \pi_\theta(v | z_t, t) - \lambda \mathcal{C}(v) \quad (1.34)$$

のように、制約コスト $\mathcal{C}$ を用いて候補トークン v の確率を抑制する形で記述できる (λ は強さ)。このような推論時制御により、要素種別条件、部分補完、属性条件など複数の条件付きタスクを単一の拡散モデルで統一的に扱えることを示した。

LayoutDiffusion: zhang ら [zhang'layoutdiffusion'2023] による LayoutDiffusion [empty citation] は、レイアウトを異種混在 (heterogeneous) な離散トークン列として表現し、離散拡散過程で生成する点では LayoutDM と同系統である。一方で、カテゴリ (nominal) と座標 (ordinal) が混在する場合、前向き遷移を一様混合のように設計すると近傍ステップ間の差分が過大になり、復元 (denoise) が困難になることを指摘する。そこで同手法は、穏やかな (**mild**) 拡散過程を実現するための設計原理として *legality* (不正トークン遷移の回避), *coordinate proximity* (座標近傍性の保持), *type disruption* (カテゴリ攪乱の制御) を掲げ、それを満たす前向き遷移行列の構成を与える。

(i) **ブロック単位の forward 遷移行列** LayoutDiffusion では、時刻 t の遷移行列を、座標 (ordinal) とカテゴリ (nominal) 等を分離したブロック構造として設計する：

$$Q_t = \begin{bmatrix} Q_t^{\text{coord}} & 0 & 0 \\ 0 & Q_t^{\text{type}} & 0 \\ 0 & 0 & Q_t^{\text{spec}} \end{bmatrix}. \quad (1.35)$$

この形により、座標トークンがカテゴリトークンへ遷移する等の明らかに不正な遷移を排除し (*legality*), 属性ごとに適切なノイズ設計を行える。

(ii) **座標 (ordinal) 用 : 離散ガウス近傍遷移** 座標トークンは順序性を持つため、現在値 k の近傍へ高確率で遷移する離散化ガウス分布に基づく遷移が用いられる。具体的には、座標語彙サイズを M とし、標準偏差 (ノイズ強度) σ_t を用いて、

$$Q_{t,k,l}^{\text{coord}} = \begin{cases} \frac{P(l) - P(l-1)}{P(M-1) - P(-1)}, & l = 0, \\ \frac{P(l) - P(l-1)}{P(M-1) - P(-1)}, & 0 < l < M-1, \\ \frac{P(M-1) - P(M-2)}{P(M-1) - P(-1)}, & l = M-1, \end{cases} \quad (1.36)$$

のように、累積確率 $P(\cdot)$ (連続ガウスの CDF を離散区間に積分したもの) を用いて近傍遷移を定義する。この設計により、小さな t では「少しだけ」座標が揺らぎ、大きな t では徐々に広がるという *coordinate proximity* を満たすノイズスケジュールを構成できる。

(iii) **カテゴリ (nominal) 用 : 吸収状態による type disruption 制御** カテゴリ属性は順序を持たないため、他カテゴリへの一様遷移を許すと意味が急激に破壊される。LayoutDiffusion はこれを避けるため、カテゴリを他カテゴリへ直接遷移させず、確率 p_t で MASK へ遷移する吸収型の遷移を導入する：

$$Q_t^{\text{type}} = \begin{bmatrix} (1-p_t)I_K & p_t\mathbf{1}_K \\ \mathbf{0}_K^\top & 1 \end{bmatrix}. \quad (1.37)$$

ここで p_t は時刻依存の攪乱強度であり, type disruption を直接制御する役割を担う. この結果, カテゴリの意味を保ちながら不確実性のみを段階的に増やすことができ, denoise の難易度を緩和できる.

(iv) 逆過程と推論時制約 逆過程は, Transformer 等により $p_\theta(z_{t-1} | z_t)$ を近似し, T ステップの反復で $z_T \rightarrow z_0$ を復元する. また LayoutDiffusion も, 推論時のトークン固定や候補抑制により条件付き生成へ拡張可能であることが示されている. これらの設計により, PubLayNet や RICO 等で既存法を上回る性能が報告され, 前向き過程の設計が離散拡散レイアウト生成の鍵であることを示した.

1.1.3 大規模言語モデルを用いたレイアウト生成

前節までで述べた深層生成モデルは, レイアウトを座標やサイズの数値列として扱い, その確率分布を学習するアプローチが主流であった. これに対し近年, 自然言語処理分野における大規模言語モデル (Large Language Models; LLMs) の発展に伴い, レイアウト生成においてもレイアウトを言語的・記号的な表現へと変換し, LLM の持つ知識や推論能力を活用する試みが活発化している [layoutprompter, layoutgpt, layoutnuwa, posterlama, tanaka'scipostlayout'2024].

数値生成から言語生成への転換 GAN, VAE, Transformer, 拡散モデルといった従来手法では, レイアウトは主に

$$(c_i, x_i, y_i, w_i, h_i)$$

のような数値属性の集合あるいは系列として表現され, モデルはこれらの値を直接予測してきた. この枠組みは, 幾何的整合性の学習には有効である一方,

- 条件や制約の設計が人手に依存する
- 新しい制約形式への拡張が難しい
- 意味的意図を明示的に扱いにくい

といった課題を抱えていた [shi'understanding'2023, huang'survey'2023].

一方, LLM は自然言語を通じて, 階層構造, 優先度, 役割分担といった抽象概念を柔軟に扱う能力を有する. この特性に着目し, レイアウト生成を言語生成問題あるいは言語を介した構造生成問題として再定式化する研究が提案されている.

LayoutPrompter LayoutPrompter [layoutprompter] は, LLM をレイアウト生成に利用する代表的研究である. この手法では, 要素種別や要素間関係といった制約を自然言語プロンプトとして記述し, LLM によってレイアウト仕様を生成させる.

例えば,

“A large title should be placed at the top, with an image below it, and text blocks aligned under the image.”

のようなプロンプトから, レイアウト要素の構造記述を生成し, それを後段の数値生成器に変換する. この設計により, 制約設計を数式や専用フォーマットではなく, 人間にとって直感的な言語表現で記述できる点が特徴である.

LayoutGPT LayoutGPT [layoutgpt] は, レイアウト生成を完全に言語生成問題として扱う点で一歩踏み込んだ手法である. この研究では, レイアウトを HTML や JSON 形式のテキストとして表現し, LLM によって直接生成する.

例えば,

```
<div class="container">
  <h1 style="top:0px">Title</h1>
  <img style="top:80px"/>
  <p style="top:300px"/>
</div>
```

のような構造化テキストを生成対象とすることで, LLM が持つコード生成能力を活用してレイアウト構造と配置を同時に記述する.

この枠組みでは, 生成モデルは

$$p_{\theta}(T \mid S)$$

として, 条件 S (プロンプト) に基づきテキスト列 T (レイアウト記述) を生成する. ここで, 幾何情報は数値であるものの, その生成は言語トークン列の一部として扱われる.

LayoutNUWA : マルチモーダル条件付き生成 LayoutNUWA [layoutnuwa] は, LLM ベースの生成をさらに拡張し, テキスト条件だけでなく, 画像やキャンバス情報を含むマルチモーダル条件付きレイアウト生成を提案した. レイアウトは依然として言語的記述として生成されるが, 視覚特徴を条件として入力することで, 背景画像や内容との整合性を考慮した配置が可能となる.

この方向性は, 要素を抽象化することなくその要素の内容までを考慮するレイアウト生成 content-aware layout generation と LLM を融合する試みとして位置づけられる.

PosterLlama : ポスター生成における LLM の活用 PosterLlama [posterlama] は、ポスターという実応用性の高い媒体を対象に、LLM によるレイアウト生成を検討した研究である。この手法では、ポスターを HTML に変換して扱い、LLM によって構造・配置・階層関係を同時に生成する。

特に、PosterLlama は

- レイアウトを数値列ではなく文書構造として捉える
- LLM の「デザイン常識」や事前知識を活用する

点を強調しており、ポスター制作のような意味的判断が重要なタスクとの親和性を示している。

SciPostLayout SciPostLayout [tanaka'scipostlayout'2024] は、科学ポスターという特定媒体に着目し、レイアウト解析と生成のためのデータセットを整備した研究である。同研究では、LayoutFormer++ や LayoutPrompter, PosterLlama などの手法を比較し、LLM 系手法が「意味構造の理解」には優れる一方で、数値的精度や幾何的厳密性には課題を残すことを指摘している。

1.1.4 制約付き・条件付きレイアウト生成の整理

前節までは、主にレイアウト生成モデルのアルゴリズムやアーキテクチャの発展について述べた。本節では、これらの手法が具体的にどのような媒体を対象とし、どのような制約（条件）を入力として扱ってきたかという観点から、既存研究を整理する。

レイアウト生成が対象としてきたのは、雑誌表紙、Web/UI、プレゼンテーションスライド、バナー広告、ドキュメント組版など、多様なデザイン媒体における視覚要素の配置である。[scipostlayout2024, huang'survey'2023, shi'intelligent'2023]. 近年は、これらの応用領域の拡大に伴い、ユーザの意図を反映した編集可能性 (controllability) を備えることが重要視され、無条件生成 (unconditional generation) と条件付き生成 (conditional generation) を区別して整理する枠組みが広く用いられる [scipostlayout2024]. 無条件生成はデータ分布からもっともらしいレイアウトを直接サンプリングする設定であり、条件付き生成はユーザが指定する制約（条件）を満たす範囲でレイアウトを生成する設定である [scipostlayout2024].

応用領域（対象媒体）の多様化 レイアウト生成研究は、媒体固有の目的関数や制約の違いを背景として、複数のアプリケーション領域で並行して発展してきた。例えば、(1) **UI レイアウト**（モバイルアプリ等）は、タップ可能性や可読性、情報密度といった機能要件が強く、RICO を代表とするデータセットと

もに研究が進展した [deka'rico'2017, rico, gupta'layouttransformer'2021, jiang'layoutformer'2023, jiang'layoutformerpp'2023]. (2) ドキュメント組版・文書レイアウトでは、段組みや図表配置などの規則性が強く、PubLayNet などのデータセットを中心に、解析と生成の両面から研究が蓄積された [publaynet, docbank, tablebank, layoutlmv3, dit]. (3) 雑誌表紙・広告・ポスターは、視線誘導や視覚的バランスに加えて、背景画像や商品画像などキャンバスに含まれるコンテンツとの調和が重要であり、内容を考慮した (content-aware) 生成が主要な論点となる [contentgan, cgl'gan, ds'gan, radm, posterlama]. (4) プレゼンテーションスライドや科学ポスターは、文章・図表・セクションといった異種要素が混在し、強いマルチモーダル性を持つため、媒体特有のデータ整備と評価が重要となる [scipostlayout2024].

これらの媒体は共通して「要素の集合／列」を扱う一方、要素種別の語彙や制約の性質（規則性の強さ、背景画像の有無、ユーザ編集の頻度）が異なるため、条件付き生成における制約設計の整理が不可欠となる。

条件付き生成における制約の類型 既存研究では、条件付きレイアウト生成で扱う制約は大きく以下に分類できる [scipostlayout2024, shi'understanding'2023].

1. 要素種別 (Gen-T) 条件

生成する要素カテゴリ集合 $\{c_i\}_{i=1}^N$ (例: title, text, image) のみを与え、それらを配置する設定である。形式的には、条件 S_{type} を

$$S_{\text{type}} = (c_1, \dots, c_N) \quad (1.38)$$

とし、モデルは

$$p_{\theta}(L \mid S_{\text{type}}) \quad (1.39)$$

を学習する。初期の GAN/VAE 系や Transformer 系の多くは、この条件を基本形として採用する [li'layoutgan'2019, jyothi2021layoutvaestochasticscenelayout, gupta'layouttransformer'2021, kong2021blt, jiang'layoutformer'2023].

2. 要素種別 + サイズ (Gen-TS) 条件

要素種別に加えて、幅・高さなどのサイズ情報 (w_i, h_i) を条件として与える設定である。

$$S_{\text{size}} = \{(c_i, w_i, h_i)\}_{i=1}^N, \quad p_{\theta}(L \mid S_{\text{size}}) \quad (1.40)$$

のように定式化され、配置の自由度を減らすことで、実務上のワークフロー（素材サイズが先に決まる等）に近づける [jiang'layoutformerpp'2023, kong2021blt].

3. 要素間関係 (Gen-R) 条件

「A は B の上」「A は B より左」「A は B と整列」といった関係制約を条件として与える設定である．典型的には,

$$S_{\text{rel}} = \{(i, r_{ij}, j)\} \quad (1.41)$$

を用いて, 関係集合を満たすレイアウトを生成する [kikuchi'constrained'2021, jiang'layoutformerpp'2023]. これは, 人間のデザイン意図を比較的解釈可能な形で入力できる利点を持つ一方, 関係集合の設計が人手に依存しやすい.

4. 補完 (layout completion)

部分レイアウト L_{partial} を条件として, 欠損要素や欠損属性を補完する設定である.

$$p_{\theta}(L_{\text{missing}} \mid L_{\text{partial}}) \quad (1.42)$$

を学習し, インタラクティブ編集や途中状態からの生成を支援する [gupta'layouttransformer'2021, jiang'layoutformerpp'2023].

5. 洗練 (layout refinement)

既存レイアウト L を入力として, 重なりや整列の改善など, 品質を向上させたレイアウト L' を生成する設定である.

$$p_{\theta}(L' \mid L) \quad (1.43)$$

の学習として定式化され, デザイン支援ツールとしての実用性と親和性が高い [rahman2021ruite, jiang'layoutformerpp'2023].

内容考慮 (content-aware) 制約: 背景画像・テキスト内容との整合 上記の制約は主に「要素の幾何属性とカテゴリ」を中心に記述されるが, ポスターや広告などでは, 背景画像やキャンバス内容との調和 (可読性・視覚的バランス) が性能を大きく左右する. この観点から content-aware layout generation として位置づけられ, 画像特徴とテキスト内容を併用して配置を決定する試みが継続的に提案されてきた [contentgan, cgl'gan, ds'gan, radm, posterlama]. 特に PosterLlama は, レイアウト要素を数値列として扱うだけでは「意味的關係」を捉えにくいという問題意識から, HTML 形式への変換により言語モデルの知識を活用し, さらに視覚内容も入力に含めることで, 視覚的コンテキストを考慮したポスター生成を目指している [posterlama, layoutprompter, layoutgpt, layoutnuwa].

本研究との接続: 制約の設計から制約の生成へ 以上より, 条件付きレイアウト生成は, 「どの制約を与えるか」を設計して可制御性を高める方向で発展してきたと言え

る。一方、既存研究で条件として扱われる制約は、要素種別・サイズ・関係・部分レイアウトなど、多くの場合 人手で与えられる構造化条件である。本研究が対象とする漫画ネーム生成では、そもそも「制約」を脚本（物語）から獲得し、その制約を入力としてレイアウト／演出を生成する必要がある。したがって、本研究は、条件付きレイアウト生成が確立してきた枠組みを踏まえつつ、物語文脈から条件（制約）を内部的に自動生成する点に焦点を置くことで、既存の条件付き生成研究を一段上の「意味条件付き生成」へ拡張する位置づけにある..

1.1.5 本研究の位置づけ

本節で述べたように、レイアウト生成は、(1) ルール・最適化による制約充足としての定式化、(2) GAN/VAE による分布学習、(3) Transformer による系列モデリングと制約直列化、(4) 拡散モデルによる反復的洗練、という方向で発展し、多様な媒体（UI、文書組版、広告・ポスター等）へ応用が拡大してきた。とりわけ近年の手法は、要素間関係や部分補完などを同一枠組みで扱うことにより、編集可能性（**controllability**）と生成品質を両立しつつ、実デザイン作業に近い条件付き生成を実現している [jiang'layoutformerpp'2023, inoue2023layoutdm, zhang'layoutdiffusion'2023]。

しかし、既存研究で中心的に扱われてきた「条件（制約）」は、要素種別、サイズ、幾何関係、部分レイアウトなど、主として 幾何構造に関する明示的・構造化された条件である。これらの条件は、ユーザが外部から与える（あるいはデータから直接得る）ことを前提に設計されており、「なぜその配置になるべきか」という意味的根拠はモデルの外側に置かれがちであった。例えば LayoutFormer++ は、多様な制約をトークン列へ統一することで汎用な条件付き生成を可能にしたが、制約自体は依然として人手で与えられることを前提としている [jiang'layoutformerpp'2023]。また LayoutDM や LayoutDiffusion は、反復的復元により高い生成品質と条件注入の柔軟性を示した一方で、条件はやはり構造化制約（固定トークン、候補剪定など）として与えられる [inoue2023layoutdm, zhang'layoutdiffusion'2023]。

近年は LLM を用い、自然言語からレイアウト記述（HTML/JSON 等）を生成する試みも活発化している [layoutprompter, layoutgpt, layoutnuwa, posterlama]。これらは「言語で意図を与える」点で条件設計を大きく前進させたが、多くはポスターや UI のような、ユーザが意図・要件を明示的に記述できる状況を主に想定している。一方で、本研究では物語（脚本）に内在する出来事・感情・関係の推移が、コマ割りや視点、情報提示順序といったレイアウト決定に直接影響するような媒体を扱い、「物語文脈 → 制約（演出上の要請） → レイアウト」という接続そのものを主要な問題とし

て取り上げている。すなわち、レイアウト生成を「与えられた制約を満たす配置」から、物語から制約を獲得し、その制約に従って配置を生成する段階への拡張である。

以上を踏まえると、既存の条件付きレイアウト生成は

制約をいかに表現し、いかに満たして生成するか

を中心に発展してきたのに対し、本研究は

物語文脈から制約（演出・構造要件）をいかに抽出・生成し、生成モデルへ接続するか

に焦点を置く点で新規性を有する。

具体的に本研究は、脚本に含まれる出来事列・登場人物関係・会話の流れ・強調すべき情報などを手掛かりとして、(i) コマの役割（導入・転換・強調等）、(ii) 視点や情報提示の順序、(iii) 重要要素の優先度や相対配置といったレイアウトに影響する高次の条件を自動的に導出し、これをレイアウト生成へ活用する枠組みを構成する。この意味で本研究は、LayoutFormer++ 等が確立した「制約を系列として扱う」設計原理を継承しつつ [jiang'layoutformerpp'2023]、制約そのものを脚本から生成することで、従来の幾何中心の条件付き生成を物語条件付き（**narrative-conditioned**）レイアウト生成へ拡張する試みとして位置づけられる。

1.2 ビジュアルストーリーテリング

画像列から、そこに内在する出来事の連なりや文脈的关系を抽出し、言語として表現する研究は、視覚情報と言語情報を統合的に扱うマルチモーダル分野の研究の一つとして発展してきた。このような研究は、画像系列に基づく物語理解・生成という観点から、一般にビジュアルストーリーテリング（**Visual Storytelling**）としても整理されている [huang2016visual]。

本節では、その基盤技術である画像キャプション生成研究から出発し、画像系列を対象とした物語生成への拡張、さらに近年の大規模視覚言語モデル（Large Vision-Language Models; LVLM）による技術的発展までを、モデル構造および学習手法の観点から体系的に概観する。

1.2.1 画像キャプション生成

画像キャプション生成は、単一画像を入力として、その内容を自然言語文として記述するタスクであり、視覚と言語を結び付ける最も基本的な課題の一つである。深層

学習に基づく初期の手法では、畳み込みニューラルネットワーク (CNN) と再帰型ニューラルネットワーク (RNN) を組み合わせたエンコーダ・デコーダ構造が標準的な枠組みとして用いられてきた [vinyals2015show, karpathy2015deep].

CNN は入力画像 $\mathbf{I}$ を畳み込み演算によって処理し、局所的な視覚特徴から高次の意味表現を抽出する。第 l 層における特徴マップ $\mathbf{F}^{(l)}$ の各要素は、前層の特徴マップ $\mathbf{F}^{(l-1)}$ と畳み込みカーネル $\mathbf{W}^{(l)}$ を用いて、一般に次式で表される：

$$\mathbf{F}_{i,j}^{(l)} = \sigma \left(\sum_{u,v} \mathbf{W}_{u,v}^{(l)} \cdot \mathbf{F}_{i+u,j+v}^{(l-1)} + b^{(l)} \right), \quad (1.44)$$

ここで $b^{(l)}$ はバイアス項、 $\sigma(\cdot)$ は ReLU などの非線形活性化関数である。最終層の出力は、画像全体を表す特徴ベクトル $\mathbf{v} = \text{CNN}(\mathbf{I})$ として得られる。

言語生成を担うデコーダには、LSTM や GRU に代表される RNN が用いられ、時刻 t における隠れ状態 $\mathbf{h}_t$ は、直前の状態 $\mathbf{h}_{t-1}$ 、入力単語 x_t 、および画像特徴 $\mathbf{v}$ に基づいて更新される：

$$\mathbf{h}_t = \phi(\mathbf{h}_{t-1}, x_t, \mathbf{v}). \quad (1.45)$$

モデルは、正解キャプション $Y = \{y_1, \dots, y_T\}$ に対する条件付き尤度を最大化するように学習される：

$$\mathcal{L} = \sum_{t=1}^T \log P(y_t \mid y_{<t}, \mathbf{I}). \quad (1.46)$$

この枠組みにより、画像内の主要な物体や行為を言語化することは可能となったが、RNN に基づく逐次生成は、長距離依存関係の保持が難しく、複雑な構造や関係性を明示的に表現する点には限界があった。

1.2.2 画像系列からの物語生成

画像キャプション生成を拡張し、複数の画像 $\{I_1, \dots, I_N\}$ を入力として一貫した物語文を生成する課題が、ビジュアルストーリーテリングとして提案された [huang2016visual]。この課題では、各画像の内容記述に加え、画像間の時間的推移や因果関係を補完し、静的な描写を動的な物語構造へと統合する能力が求められる。

初期研究では、各画像から抽出された特徴ベクトル $\mathbf{v}_n = \text{CNN}(I_n)$ を入力とし、階層的に構成された RNN によりローカルな文脈と物語全体の文脈を分離して扱う手法が提案された [yu2017hierarchically, nahian2019hierarchical]。物語レベルの状態更新は、概念的には次式で表される：

$$\mathbf{h}_t^{\text{story}} = \phi_{\text{global}}(\mathbf{h}_{t-1}^{\text{story}}, \mathbf{v}_{\tau(t)}), \quad (1.47)$$

ここで $\tau(t)$ は、生成中の単語が対応する画像インデックスを表す。

しかし、RNN による逐次的な状態更新では、系列が長くなるにつれて情報が圧縮されやすく、物語全体の整合性を安定して維持することが困難であった。この課題に対し、自己注意機構を用いる Transformer アーキテクチャが導入され、長距離依存関係のモデリング能力が大きく向上した。

1.2.3 大規模視覚言語モデルへの発展

画像と言語を同時に扱う視覚言語モデルは、エンコーダ・デコーダ型の枠組みから、Transformer に基づく大規模事前学習へと発展する中で、表現能力と汎用性を高めてきた。特に、(1) 視覚特徴を系列として扱う表現の統一、(2) 視覚と言語を同一空間へ整合させる学習、(3) 生成モデルとして視覚条件付き言語生成を可能にする統合、という段階を経ることで、現在の大規模視覚言語モデル (Large Vision-Language Models; LVLM) へと至っている。以下では、この発展を主要なモデル構造ごとに整理する。

Vision Transformer による視覚表現の系列化

初期の視覚言語モデルでは、視覚特徴は CNN により抽出され、固定長ベクトルあるいは粗い空間特徴として言語側へ渡されることが多かった。しかし、物体間関係や広域文脈の統合には、局所演算を積み重ねる CNN のみでは限界があり、視覚側にも「系列としての表現」と「長距離依存を捉える機構」を導入することが重要になった。

dosovitskiy ら [dosovitskiy2020image] は Vision Transformer (ViT) により、画像を固定サイズのパッチに分割し、それらを系列として扱うことで、自己注意機構により画像全体の関係性を直接モデル化する手法を提案した。

入力画像 $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ を $P \times P$ のパッチに分割し、各パッチをフラット化したベクトル $\mathbf{x}_i$ に写像する：

$$\mathbf{x}_i = \text{Flatten}(\mathbf{I}_i)W_E + \mathbf{e}_i, \quad (1.48)$$

ここで W_E は線形射影行列、 $\mathbf{e}_i$ は位置埋め込みである。これらを Transformer Encoder に入力し、自己注意機構

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \quad (1.49)$$

により特徴を更新する。この枠組みにより、視覚情報も言語と同様に「系列」として扱えることが示され、後続の視覚言語統合モデルにおいて、視覚側表現を Transformer と親和的に設計する土台が整った。

対照学習による視覚と言語の表現空間統合

視覚表現を系列化できるようになったとしても、視覚特徴とテキスト特徴が「同じ意味空間」で対応づけられていなければ、両者を一貫した形で扱うことは難しい。この課題に対し、大量の画像・テキスト対を用いて対応するペアは近く、対応しないペアは遠くなるように学習する対照学習 (contrastive learning) が有効であることが示された。

視覚と言語を同一の意味空間に埋め込む手法として、CLIP [radford2021learning] や ALIGN [jia2021scaling] が提案された。これらのモデルでは、画像エンコーダ f_{img} とテキストエンコーダ f_{text} を独立に学習し、対応する画像・テキスト対 $(\mathbf{I}_i, \mathbf{T}_i)$ の埋め込み類似度を最大化する。

正規化された埋め込みを

$$\mathbf{v}_i = \frac{f_{\text{img}}(\mathbf{I}_i)}{\|f_{\text{img}}(\mathbf{I}_i)\|}, \quad \mathbf{t}_i = \frac{f_{\text{text}}(\mathbf{T}_i)}{\|f_{\text{text}}(\mathbf{T}_i)\|} \quad (1.50)$$

とすると、学習目的は InfoNCE 損失として

$$\mathcal{L} = - \sum_i \log \frac{\exp(\mathbf{v}_i^\top \mathbf{t}_i / \tau)}{\sum_j \exp(\mathbf{v}_i^\top \mathbf{t}_j / \tau)} \quad (1.51)$$

で与えられる。

この枠組みは、ゼロショット分類や検索において強力であり、「視覚と言語の意味対応」を大規模データから獲得できることを示した。一方で、CLIP/ALIGN は本質的に「整合 (alignment)」の学習であり、物語生成のような 長文生成・推論を伴う生成的タスクに対しては、別途「生成モデルとしての統合」が必要になるという課題が残った。

クロスモーダル注意による生成モデル

生成的タスクに対応するには、視覚表現を参照しながら言語を逐次生成できることが重要となる。この方向性では、視覚特徴を言語デコーダへ条件として与え、言語側の生成過程で必要に応じて視覚情報を参照する設計が採用されてきた。

生成タスクへの対応として、視覚特徴を言語生成に条件付けるモデルが提案された。BLIP [li2022blip] や Flamingo [alayrac2022flamingo] は、視覚エンコーダと大規模言語モデルを結合し、クロスモーダル注意機構によって視覚情報を言語生成に注入する。

言語デコーダにおけるクロスアテンションは、

$$\text{CrossAttn}(Q, K_V, V_V) = \text{softmax} \left(\frac{QK_V^\top}{\sqrt{d}} \right) V_V \quad (1.52)$$

として定義され、クエリ Q は言語トークン、キー・バリュー (K_V, V_V) は視覚特徴から構成される。この構造により、モデルは画像内容を参照しながら文章を生成でき、画像キャプション生成や VQA といった生成タスクが大きく前進した。

ただし、クロスアテンションで「視覚を条件として参照する」設計では、視覚と言語が依然として異なる処理経路に分かれており、多段推論や長い文脈統合の観点では、より統一的な表現・処理が望まれるようになった。

統合型 LVLM への発展

こうした背景から、視覚特徴を言語トークンと同等の単位へ変換し、単一の Transformer により処理する統合型モデルが提案されている。この設計は、視覚と言語の相互作用を「クロスアテンションでの参照」に限定せず、自己注意の枠組みで同等に扱える点に特徴がある。

近年では、視覚特徴を言語トークンと同等に扱い、単一の Transformer により処理する統合型 LVLM が提案されている。LLaVA [liu2023llava] などは、視覚エンコーダで得た特徴 $\mathbf{V}$ を、線形射影により言語埋め込み空間へ写像し、

$$\tilde{\mathbf{V}} = \mathbf{V}W_P \quad (1.53)$$

として得られた視覚トークン列を、テキストトークン列と連結することで、

$$\mathbf{X} = [\tilde{\mathbf{V}}_1, \dots, \tilde{\mathbf{V}}_M, \mathbf{T}_1, \dots, \mathbf{T}_N] \quad (1.54)$$

という単一系列として入力し、自己注意により統合的に処理する。この枠組みは、視覚と言語を跨ぐ推論や長い文脈統合を促進し、物語生成や複雑な指示理解といった高次タスクにおける性能向上につながっている。（なお、GPT-4V などの商用モデルも同様に、視覚情報をトークン化して統合する方向性に基づく）と理解できる [wu2024gpt4o].）

1.2.4 Visual Prompting

大規模視覚言語モデル（Large Vision-Language Models; LVLM）は、自然画像を対象とした事前学習により、画像理解・推論・言語生成において高い性能を示している。一方で、特定ドメインに固有の視覚構造や記号体系を持つ入力に対しては、モデルのパラメータを直接更新する fine-tuning を行わずに適応させることが求められる場合も多い。

このような背景のもと、入力の与え方を工夫することでモデルの推論挙動を制御する枠組みとして、**Prompting**、およびその視覚領域への拡張である **Visual**

Prompting が注目されている．本節では，Prompting の概念的背景から，視覚タスクにおける Visual Prompting の発展に付いて整理する．

Prompting と In-Context Learning

Prompting は，大規模言語モデル（LLM）に対して，明示的なパラメータ更新を行うことなく，入力文（プロンプト）の設計によってタスク遂行を誘導する手法である．特に GPT-3 [brown2020language] によって示された **In-Context Learning** は，少数の例示や指示文を入力に含めるだけで，新たなタスクに適応可能であることを明らかにした．

数理的には，Prompting は，モデルが学習済みの条件付き確率分布

$$P(y \mid x) \quad (1.55)$$

に対し，追加の文脈 p を与えることで，

$$P(y \mid x, p) \quad (1.56)$$

として再条件付けを行う操作とみなすことができる．ここで x は入力， p はプロンプト， y は出力である．この枠組みは，パラメータ空間ではなく入力空間を操作することで推論挙動を制御する点に特徴がある．

視覚タスクにおける Prompting の拡張

Prompting の概念は，言語モデルに限らず，視覚タスクにも拡張されてきた．その初期例として，セグメンテーションや検出において，点，バウンディングボックス，スクリブルなどの人為的な視覚的手がかりを入力として与える研究が挙げられる [xu2016deepgrabcut, maninis2018deep]．

これらの手法では，視覚的手がかり p_{vis} を追加することで，推論対象領域を明示的に制約し，

$$P(y \mid x) \rightarrow P(y \mid x, p_{\text{vis}}) \quad (1.57)$$

という条件付き推論を実現している．この考え方は，モデルの能力そのものを変更するのではなく，探索空間を縮小するという点で，Prompting と本質的に共通している．

Visual Prompting の定義と特徴

Visual Prompting は，この入力操作の考え方を，視覚言語モデルに体系的に適用する枠組みである．Chen ら [chen2023visualprompting] は，Visual Prompting を「モデルのパラメータを固定したまま，入力画像に視覚的変換を施すことで，下流タスクへの適応を実現する手法」と定義している．

代表的な Visual Prompting の形式には、以下が含まれる：

- 画像へのマーキング（枠線，番号，強調表示）
- マスクやオーバーレイの付与
- 空間的構造を明示する補助情報の重畳

これらは、画像 x に対して視覚的プロンプト p_{vis} を付与し、

$$x' = \mathcal{T}(x, p_{\text{vis}}) \quad (1.58)$$

という変換を施した上で、モデルに入力する操作と捉えられる．ここで $\mathcal{T}$ は視覚的変換操作である．

LVLM における Visual Prompting の役割

LVLM は、視覚トークンとテキストトークンを統合的に処理する能力を持つ一方、入力画像内のどの情報を推論に用いるべきかについては、必ずしも明示的な誘導を持たない．そのため、複雑な構造や非自然画像的な表現を含む入力では、推論が不安定になることが報告されている [vivoli2025comix]．

Visual Prompting は、この問題に対し、モデルの注意機構が参照すべき情報を視覚的に明示するという役割を果たす．すなわち、Visual Prompting は、LVLM の能力を拡張するというよりも、既存の能力を適切に引き出すためのインターフェース設計として位置づけられる．

本研究における位置づけ

ビジュアルストーリーテリング研究は、画像系列を入力として、その内容に内在する出来事の推移や文脈関係を捉え、自然言語による物語文を生成することを主目的として発展してきた．特に近年では、Transformer や LVLM の導入により、長距離依存関係のモデリングや高次の意味推論が可能となり、生成される物語文の一貫性や表現力は大きく向上している．

一方で、従来のビジュアルストーリーテリングにおける出力は、主として「人が読むこと」を想定した散文的な物語文であり、その内容を後段の生成・設計工程に直接利用することは必ずしも想定されてこなかった．すなわち、物語としての自然さや流暢さは重視されてきた一方で、出来事や関係、役割分担といった情報を明示的に整理された構造化表現として出力する取り組みは十分に組み込まれてきたとは言えない．しかし、漫画のネームを構築することを考慮すると、物語理解の結果を、自由形式の文章に加えて、編集支援、演出設計、配置設計などに有用な要素、順序、関係が明示された中間表現として整理されることが望ましい．

本研究は、このギャップに着目し、ビジュアルストーリーテリングを「画像系列から物語文を直接生成する問題」としてではなく、画像系列から、後段で利用可能な物語構造を抽出する問題として再整理する立場を取る。すなわち、最終的な自然言語文の生成そのものよりも、出来事、登場要素、関係性、進行順序といった物語を構成する情報を明示的に整理した構造化出力を獲得することを目的とする。

さらに、このような構造化出力を安定して得るためには、モデルが入力画像のどの情報に着目すべきかを適切に誘導することが重要となる。本研究では、Visual Prompting の枠組みを用いて、入力の与え方を工夫することで、LVLM の注意を物語構造の抽出に有効な視覚要素へと誘導する。これは、モデルのパラメータを変更することなく、入力設計によって推論挙動を制御するという点で、既存の Prompting / Visual Prompting 研究の考え方と整合的である。

以上より、本研究は、ビジュアルストーリーテリング研究の流れを踏まえつつ、その出力を「読むための物語」から「生成・設計に接続するための構造化表現」へと拡張する点に新たな位置づけを持つ。

1.3 漫画を対象とした研究

漫画は、コマ割りによる時間・空間の離散化、吹き出し内テキストの高密度な配置、誇張表現や記号表現（漫符）など、自然画像とは異なる視覚言語体系の上に成立する媒体である。そのため、漫画を対象とした画像理解・物語理解研究では、一般物体認識や自然画像キャプション生成で前提とされる「写実的外観」「連続的時間」「単一視点」を仮定できず、媒体固有の構造を明示的に扱う必要がある。

本節では、(i) 漫画を対象とする代表的データセット、(ii) 漫画読解が難しい理由（多段タスク構造）と、それを評価する近年のベンチマーク、の順に整理し、本研究が扱う「脚本に基づくネーム生成」へと接続する。

1.3.1 漫画理解を支えるデータセット

漫画理解研究を推進する上で、コマ・吹き出し・テキスト・キャラクターといった構造の注釈付きデータは不可欠である。とりわけ漫画は、作品・作者・掲載誌により画風や記号表現が大きく変化するため、多様なスタイルを含むデータ整備は、汎用的な読解能力の獲得と評価の前提となる。

eBDtheque Guérin ら [eBDtheque: A Representative Database of Comics] による eBDtheque は、合計 100 ページの漫画画像からなるデータセットである。例

例えばバージョン 3 では、100 ページに対し、850 のコマ領域、1081 のセリフ吹き出し領域、4,693 行のセリフ文字列、1,620 のキャラクター描画といった粒度で整備されている。

COMICS 大規模データとしては、Iyyer ら [iyyer2017comics] の *COMICS* が代表的であり、これは 120 万以上のコマに対して自動のテキスト認識を付与したデータセットである。同データは、複数パネル文脈に基づくテキストや視覚の穴埋めタスクを通じ、「コマ間の欠落を推論して物語を接続する」能力を評価するベンチマークを築いている。

Manga109Dataset 日本語漫画に特化した代表的データセットとして、Matsui ら [matsui2017manga109] による Manga109Dataset がある。Manga109Dataset は、1990 年代から 2000 年代にかけて出版された日本の商業漫画 109 冊を収録し、著作者許諾のもと研究利用を目的に公開されている合計 21,142 ページからなる大規模な漫画画像データセットである。少年漫画・少女漫画・青年漫画など異なるジャンルが網羅的に収録され、特定の作品・作家・画風に依存しない汎用的な漫画理解の獲得に適したデータであるといえる。各ページには複数のアノテーションが付与されており、キャラクター領域、セリフ領域、コマ領域のバウンディングボックスに加え、セリフ文字列や話者 ID など整備されている。図 1.1 に Manga109Dataset の例を示す。

Manga109 派生データ：物語文・属性注釈への拡張 Manga109 を基盤として、擬音語や漫符など漫画特有表現に関する研究も進められており、例えば、対話理解の基盤整備として、セリフテキストと話者キャラクター（バウンディングボックス）を対応付けた大規模注釈データセット Manga109Dialog が提案されている。Manga109Dialog は話者-テキスト対応を 132,692 ペア規模で整備し、同一コマ内に話者が存在しない難例も含めて漫画における話者推定の評価基盤（ベンチマーク）を提供する [li2024manga109dialog]。また近年は、ページ画像に対して物語文を付与する方向の整備も進んでおり、Manga109Story は Manga109 の一部に対してストーリー記述データを構築する試みとして報告されている [chen2024manga109story]。ただし、Manga109Story の物語文は散文的記述であり、本研究が意図する実制作上の「脚本形式の記述」とは目的・粒度が異なる点に留意が必要である。

1.3.2 漫画読解の多段構造と評価ベンチマーク

近年の大規模視覚言語モデル（Large Vision-Language Models; LVLM）の発展により、自然画像を対象とする VQA やキャプション生成、画像推論は著しい性能向上を示している。一方で漫画読解は、二次元レイアウト構造と文字情報と物語推論が強



図 1.1: Manga109Dataset に含まれる漫画画像の例 (『ARMS』より)。© 加藤 雅基／東京三世社

く結合しており，単一画像・単一視点を前提とする一般的な視覚理解の枠組みでは扱いにくい．とりわけ，漫画を「読む」行為は単一タスクでは完結せず，複数の処理を段階的に積み上げる必要がある点に本質的な難しさがある．

多段タスクとしての漫画読解 漫画読解に必要な処理は，典型的には以下のサブタスク群として整理できる．これらは独立ではなく，上流の誤りが下流に伝播する (error propagation) ため，全体としての頑健な読解を難しくする．

- **テキスト検出と意味理解**：セリフや効果音などの言語情報を抽出し内容を解釈することは，物語内容を把握するうえで漫画の読解に不可欠である．
- **読解順序の推定**：テキスト情報は単体ではなく物語の流れの中で意味を持つため，会話の流れや因果関係を正しく捉えるうえで読解順序を推定することは漫画の読解に不可欠である．
- **話者推定・対応付け**：各発話を適切な話者へ対応付けられなければ，意図された人物関係や出来事理解が崩れるため，話者推定は漫画の読解に不可欠である．
- **キャラクター同一性認識**：デフォルメや部分遮蔽が頻発する条件下でも同一キャラクターを同定・追跡することは，行動や視点の連続性を把握するうえで

漫画の読解に不可欠である。

このように漫画読解は、複数のサブタスクの同時要求として理解でき、LVLM の進歩にもかかわらず依然として困難なタスクとして残っている。

漫画理解におけるベンチマーク整備 このような漫画の読解困難性を示す定量的な死票として近年では漫画理解に必要な能力を細分化し評価するベンチマークが整備されつつある。MangaUB [ikuta2024mangaub] は、単一パネルの内容認識に閉じない評価を目指し、複数パネルをまたいだ理解を含む多面的な評価枠組みを提示している。これにより、最新の LVLM のモデルでも「認識としては強いが、複数パネルにまたがる感情・情報の理解は難しい」といった弱点を可視化している。

より包括的な評価の方向として、CoMix [vivoli2024comix] は、検出（パネル・人物・テキスト等）、対応付け（話者同定・キャラクター再同定等）、読解順などの計算的タスクに加え、「誰が何を言ったか」を順序付きで書き起こすような統合タスクまで含む総合ベンチマークを提示している。ベースライン評価では LVLM と人手との差が大きいことが報告されており、漫画読解が依然として未解決のギャップとして残っていることを示している。

Manga109Dialog [li2024manga109dialog] は、従来の「セリフに最も近い人物が話者」という規則ベースが複雑な例で破綻しうることを指摘し、人物領域とテキスト領域の関係を明示的に扱う大規模データセットとベンチマークを整備している。ここでも、読解順序やコマ（フレーム）といった漫画固有の構造を考慮しないと、安定した推定が難しいことが示唆される。

推論時設計（プロンプト／パイプライン）による押し上げ 一方で、このギャップを直接埋める方向として、既存 LVLM を追加学習で作り替えるのではなく、推論時の工夫により漫画理解を押し上げる試みも現れている。ComiCap [vivoli2024comiccap] はその代表例であり、既存の Vision-Language Models (VLMs) を用いてコミックパネルの dense captioning を生成するパイプラインを提案している。同研究は、単に自由文キャプションを生成するのではなく、出力を「属性を保持した記述」へ誘導し、さらに **grounding**（記述と領域の対応付け）まで含めて構造化するという設計を取ることで、漫画の内容理解を段階的に外在化する方針を示している [vivoli2024comiccap]。この観点は、多段タスク構造を「一発で解く」のではなく、何を出力すべきかを明示し、途中表現を構造化して積み上げることで全体の頑健性を高める戦略として重要である。

本研究との接続 以上より，漫画読解は多段タスクであり (MangaUB, CoMix)，特に複数パネル統合や対応付けで困難が顕在化する．本研究はこの「難しいギャップ」を前提として，LVLM を基盤にしつつ，(1) キャラクター識別子 (ID) の付与による同一性の明示，(2) 読解順序の明示的付与，(3) 段階的な問い・入力設計 (Visual Prompting) による中間表現の構造化，を通じて，多段構造の誤り伝播を抑えながら，漫画画像から物語文および演出情報を構造化して抽出し，脚本に基づくネーム生成へ接続することに取り組んだ．